AIR UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE

AEROSPACE & AVIATION CAMPUS, KAMRA

PharmaEase

Enterprise Pharmacy Management System

Software Requirements Specification

Prepared By

Wasif Ali *Reg#: 245187*

Anisa Khan *Reg#: 245215*

"Connecting Patients to Medicines — Reliably, Efficiently, Securely"

Abstract

PharmaEase is an enterprise-grade, desktop-based Pharmacy Management System architected in **Java** and strictly governed by the **Model-View-Controller (MVC)** design paradigm. Supported by a highly normalized **MySQL** relational database backend, the system is designed to completely digitise, automate, and streamline core pharmacological workflows. By transitioning away from conventional paper-based ledgers, PharmaEase directly mitigates human error, improves inventory accuracy, guarantees secure data handling, and significantly enhances the operational efficiency of multi-seller pharmacy environments.

This Software Requirements Specification (SRS) is drafted in rigorous compliance with the **IEEE 830-1998 standard**. It provides an exhaustive decomposition of the software's constraints. It spans operational scope, in-depth functional and non-functional requirements, detailed use-case workflows, external interfaces, and advanced system modelling. The document incorporates high-fidelity UML artefacts—including Architecture, Use Case, Activity, Sequence, Class, Domain, and Entity-Relationship Diagrams. All documentation has been reverse-engineered directly from the final Java codebase to guarantee absolute structural parity between the written blueprint and the deployed software product.

Executive Summary: This document serves as the absolute source of truth for the PharmaEase project. It details the secure authentication protocols, multi-threaded inventory handling mechanisms, and strict role-based access controls designed, developed, and tested by Wasif Ali and Anisa Khan at Air University, Kamra.

Contents

Abstract **1**

1 Introduction **5**

1.1 Purpose 5

1.2 Document Scope and Organization 5

1.3 Product Scope and Business Context 6

1.4 Definitions, Acronyms, and Abbreviations 6

1.5 References 7

2 Overall Description **8**

2.1 Product Perspective 8

2.2 User Classes and Characteristics (Personas) 8

2.2.1 Administrator (Super User) 8

2.2.2 Seller / Pharmacist (Standard User) 9

2.2.3 Customer (End User) 9

2.3 Operating Environment 9

2.4 Design and Implementation Constraints 10

2.5 Assumptions and Dependencies 10

3 Detailed Functional Requirements **11**

3.1 FR Group 1: User Management & Authentication 11

3.1.1 FR 1.1: Multi-Role Account Registration 11

3.1.2 FR 1.2: Secure Login and Role Dispatch 12

3.2 FR Group 2: Medicine & Inventory Operations 12

3.2.1 FR 2.1: Global Medicine Catalog Management 12

3.2.2 FR 2.2: Batch-Level Inventory Tracking 13

3.3 FR Group 3: Order Processing & Checkout 13
3.3.1 FR 3.1: Persistent Shopping Cart Operations 14
3.3.2 FR 3.2: Transactional Order Placement 14
3.3.3 FR 3.3: Legal Prescription Validation 15

4 Data Dictionary **16**
4.1 Table: `users` 16
4.2 Table: `medicines` 16
4.3 Table: `inventory` 17
4.4 Table: `orders` and `order_items` 18

5 Non-Functional Requirements **19**
5.1 Performance & Scalability Metrics 19
5.2 Security & Access Control 19
5.3 Reliability and Robustness 20
5.4 Maintainability and Code Quality 20

6 System Interfaces **21**
6.1 User Interfaces (UI) 21
6.2 Software Interfaces 21
6.3 Hardware Interfaces 22

7 System Modeling and Diagrams **23**
7.1 System Architecture Diagram 23
7.2 Use Case Diagram 24
7.3 Domain Model 25
7.4 Activity and Class Diagrams 26
7.5 Sequence Diagram 27
7.6 Entity-Relationship Diagram (ERD) 28

8 Testing & Quality Assurance **29**
8.1 Testing Strategy 29
8.2 Detailed Test Case Validation Matrix 29

9 Requirements Traceability Matrix **31**

10 Conclusion **32**

A User Interface (UI) Screenshots **33**

B Core Code Snippets **35**

B.1 Database Connectivity (Singleton Pattern) 35

B.2 User Controller (Authentication Logic) 36

List of Figures

7.1 Three-Tier System Architecture (MVC Pattern) 23

7.2 PharmaEase System Use Case Diagram 24

7.3 PharmaEase Conceptual Domain Model 25

7.4 Activity Workflow and Static Class Structure 26

7.5 System Sequence Diagram (Authentication & Checkout Flow) 27

7.6 Relational Database Schema (ERD) 28

A.1 PharmaEase Seller Dashboard 33

A.2 PharmaEase customer Dashboard 33

A.3 PharmaEase Admin Dashboard 34

1. Introduction

1.1 Purpose

The purpose of this document is to formally specify the software requirements for **PharmaEase**, a comprehensive Pharmacy Management System (PMS). It serves as a binding contract between the development team and the project stakeholders (academic supervisors). This SRS guarantees that developers, database administrators, and software testers possess a unified, unambiguous understanding of the software's capabilities, technical constraints, and operational environment before final evaluation.

1.2 Document Scope and Organization

This SRS systematically details the complete lifecycle requirements of the PharmaEase application.

- **Chapters 1 & 2:** Define the business context, regulatory requirements, user profiles, and operational boundaries of the software.
- **Chapter 3:** Provides an exhaustive breakdown of the functional requirements, formatted as testable Use Case narratives.
- **Chapter 4:** Details the Data Dictionary, outlining every database table, column, and relationship.
- **Chapter 5 & 6:** Outline non-functional requirements (security, performance) and system interfaces.
- **Chapter 7:** Presents the structural and behavioral UML diagrams that map the system's architecture.

- **Chapter 8 & 9:** Dictate the testing strategies, test cases, and the Requirements Traceability Matrix (RTM).

1.3 Product Scope and Business Context

PharmaEase is conceived as a **standalone, multi-role desktop application** operating on a local area network (LAN) or single-machine environment. The primary business objective is to eliminate the inefficiencies, stock miscalculations, and security risks associated with manual pharmacy bookkeeping.

The software systematically handles three distinct operational domains:

1. **Administrative Domain:** Offers system-wide oversight. Administrators possess global management privileges to curate the central medicine catalog, categorize pharmaceuticals, audit user accounts, and review aggregate financial metrics.
2. **Seller Domain:** Provides certified pharmacists with an isolated Point-of-Sale (POS) and inventory management interface. Sellers track specific medical batches, monitor expiry dates, and process customer orders dynamically without interfering with system-level configurations.
3. **Customer Domain:** Delivers an intuitive storefront. End-users can securely browse the medical catalog, manage a persistent shopping cart, execute checkouts, and submit digital prescriptions required for restricted or controlled substances.

1.4 Definitions, Acronyms, and Abbreviations

Term	Definition
MVC	Model-View-Controller: An architectural pattern separating UI (View), logic (Controller), and data (Model).

JDBC	Java Database Connectivity: The standard Java API for establishing connections to relational databases.
RBAC	Role-Based Access Control: A security paradigm restricting system access based on the logged-in user's role.
POJO	Plain Old Java Object: Pure data structures used in the Model layer, containing private fields and public getters/setters.
CRUD	Create, Read, Update, Delete: The four fundamental operations of persistent storage.
ERD	Entity-Relationship Diagram: A visual representation of the database schema and foreign key relationships.
RTM	Requirements Traceability Matrix: A document mapping functional requirements to specific test cases.
GUI	Graphical User Interface: The visual components (Java Swing) the user interacts with.
SQL Injection	A code injection technique that might destroy a database; prevented in this project via `PreparedStatements`.

1.5 References

1. IEEE Computer Society. (1998). *IEEE Recommended Practice for Software Requirements Specifications* (IEEE Std 830-1998).

2. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

3. Oracle Corporation. (2023). *Java Platform, Standard Edition 17 API Specification*.

4. MySQL AB. (2023). *MySQL 8.0 Reference Manual*.

2. Overall Description

2.1 Product Perspective

PharmaEase operates as an independent, self-contained desktop system. However, its architecture simulates a highly scalable enterprise environment by strictly isolating the Java Swing Graphical User Interface (GUI) from the MySQL persistence layer. The `DBConnection` class utilizes the Singleton design pattern to ensure optimal network resource management, mitigating the risk of database lockouts, memory leaks, and connection pool exhaustion during simultaneous multi-user transactions.

2.2 User Classes and Characteristics (Personas)

To maintain strict data security and operational integrity, the system divides user access into three rigid, mutually exclusive tiers.

2.2.1 Administrator (Super User)

- **Technical Expertise:** High. Understands database concepts and system management.
- **Privileges:** Unrestricted CRUD access to global tables (Users, Medicines, Categories).
- **Responsibilities:** The Admin is responsible for onboarding new medicines into the global catalog to prevent data duplication. They monitor system health, oversee seller activities, and handle user dispute resolutions.

2.2.2 Seller / Pharmacist (Standard User)

- **Technical Expertise:** Medium. Comfortable with standard Point-of-Sale (POS) systems.
- **Privileges:** Confined entirely to inventory management and order processing for their specific assigned ID.
- **Responsibilities:** Adding physical stock batches to the database, ensuring expiry dates are accurately logged, and processing incoming orders routed to their pharmacy.

2.2.3 Customer (End User)

- **Technical Expertise:** Low to Medium. Requires a highly intuitive, visually guided interface.
- **Privileges:** Can only view the global catalog, place personal orders, and view their own purchase history.
- **Responsibilities:** Searching for medicines, maintaining their shopping cart, uploading valid prescriptions for controlled substances, and executing secure checkouts.

2.3 Operating Environment

The software is designed to be highly portable across modern operating systems.

- **Client Platform:** Java Runtime Environment (JRE) Version 17 or higher. Fully cross-platform (Windows 10/11, macOS 12+, Linux Ubuntu 20.04+).
- **Database Server:** MySQL Community Server Version 8.0 or higher.
- **Development IDE:** Eclipse IDE for Enterprise Java.
- **Hardware Minimums:** 4GB RAM, Intel Core i3 Processor (or AMD equivalent), 500MB available disk space, 1280×720 minimum display resolution.

2.4 Design and Implementation Constraints

- **UI Limitations:** Java Swing relies on absolute (`null`) layouts in this specific application to ensure pixel-perfect rendering of the complex dashboard elements. Consequently, dynamic window resizing is disabled to prevent UI component overlapping.
- **Database Dependency:** The application is purely a client; it cannot operate offline. A persistent TCP/IP connection to the localhost (or remote) MySQL instance is mandatory for all operations.
- **Storage Limitations:** Prescriptions are currently handled via textual input (Doctor Name, Date, Notes) rather than physical image/PDF uploads due to BLOB storage constraints in the current prototype phase.

2.5 Assumptions and Dependencies

1. It is assumed that the host machine running the software has the MySQL service actively running on default port 3306.
2. It is assumed that the `mysql-connector-java-8.x.jar` driver is correctly configured in the Java Build Path.
3. It is assumed that the database schema (`pharmacy_management.sql`) has been successfully imported and executed prior to launching the `PharmaEase.java` entry point.
4. At least one default Administrator account must be hard-coded or manually inserted into the database to allow initial system configuration.

3. Detailed Functional Requirements

This section outlines the precise, testable functional requirements of the PharmaEase system. They are structured as detailed Use Case specifications to provide absolute clarity to the development and testing teams.

3.1 FR Group 1: User Management & Authentication

3.1.1 FR 1.1: Multi-Role Account Registration

- **Description:** The system shall provide a unified graphical interface for new users to register as either a Customer or a Seller.
- **Pre-conditions:** The MySQL database is active, and the `users` table is accessible.
- **Normal Flow:**
  1. User selects "Sign Up" from the main splash screen.
  2. User enters standard details: Username, Password, Email, Phone, Address.
  3. User selects their desired Role (Customer or Seller) via a dropdown or radio button.
  4. If "Seller" is selected, the GUI dynamically reveals additional mandatory fields: "Pharmacy Name" and "License Number".
  5. User clicks Submit.
  6. The `UserController` verifies that the chosen Username does not already exist.
  7. System executes an `INSERT` into the core `users` table, retrieves the generated `user_id`, and executes a secondary `INSERT` into the specific role table (`customers` or `sellers`).

- **Alternative Flow:** If the Username exists, a `JOptionPane` error is displayed, and the process halts.
- **Post-conditions:** A new user record is permanently established in the database.

3.1.2 FR 1.2: Secure Login and Role Dispatch

- **Description:** The system must authenticate credentials and route the user to their specific, restricted dashboard.
- **Normal Flow:**
  1. User inputs Username and Password.
  2. `UserController.validateLogin()` executes a `PreparedStatement` querying the database.
  3. If credentials match, the system retrieves the `user_type_id`.
  4. The current `LoginPage JFrame` is disposed of.
  5. Based on the ID, the system instantiates either `AdminDashboard`, `SellerDashboard`, or `CustomerDashboard`.
- **Post-conditions:** A secure session is initiated within the bounds of the user's RBAC privileges.

3.2 FR Group 2: Medicine & Inventory Operations

3.2.1 FR 2.1: Global Medicine Catalog Management

- **Description:** Administrators alone possess the right to define new medicines in the system to prevent Sellers from creating duplicate, conflicting records of the same drug.
- **Pre-conditions:** User is authenticated as an Admin.

- **Normal Flow:**
  1. Admin accesses the "Manage Medicines" tab.
  2. Admin inputs Name, Manufacturer, Price, Category ID, and toggles the `RequiresPrescription` boolean flag.
  3. System validates inputs (e.g., Price > 0).
  4. System performs `INSERT INTO medicines`.
- **Post-conditions:** The new medicine immediately populates in the search grids for all Customers and Sellers globally.

3.2.2 FR 2.2: Batch-Level Inventory Tracking

- **Description:** Sellers must add physical stock to the system. Stock is tracked by unique batch numbers to monitor expirations.
- **Normal Flow:**
  1. Seller selects a globally defined Medicine from the catalog.
  2. Seller inputs a unique Batch Number, Quantity received, Manufacturing Date, and Expiry Date.
  3. System ensures the Expiry Date is chronologically after the Manufacturing Date and current system date.
  4. System executes `INSERT INTO inventory`.
- **Post-conditions:** The specific Seller's inventory is increased. Customers can now purchase this medicine.

3.3 FR Group 3: Order Processing & Checkout

3.3.1 FR 3.1: Persistent Shopping Cart Operations

- **Description:** Customers must be able to browse available inventory and build a prospective order.
- **Normal Flow:**
  1. Customer views available medicines in a `JTable`.
  2. Customer selects an item, enters a quantity, and clicks "Add to Cart".
  3. System verifies that the requested quantity is $\leq$ the total available stock in the `inventory` table.
  4. The item is added to the in-memory cart, and the sub-total and grand total fields are dynamically updated.

3.3.2 FR 3.2: Transactional Order Placement

- **Description:** The checkout mechanism must be atomic and fail-safe to prevent database corruption.
- **Normal Flow:**
  1. Customer initiates Checkout.
  2. System invokes `java.sql.Connection.setAutoCommit(false)` to begin a transaction block.
  3. System creates the header record: `INSERT INTO orders`.
  4. System loops through cart items, executing `INSERT INTO order_items`.
  5. System executes `UPDATE inventory SET quantity = quantity - requested_qty`.
  6. If all steps succeed, `Connection.commit()` is executed.
- **Exception Flow:** If a SQL error occurs (e.g., another user bought the last item milliseconds prior), an `SQLException` is caught, `Connection.rollback()` is executed, and the user is notified.

3.3.3 FR 3.3: Legal Prescription Validation

- **Description:** The system must legally restrict the sale of controlled substances unless an authorisation record exists.
- **Normal Flow:**
  1. During checkout initiation, the system iterates over the cart.
  2. If any `Medicine` object has `requires_prescription == 1`, checkout is paused.
  3. A modal window forces the Customer to input prescription details (Doctor Name, Issue Date).
  4. These details are inserted into the `prescriptions` table.
  5. The newly generated `prescription_id` is retrieved and permanently linked as a Foreign Key to the `orders` table during final placement.

4. Data Dictionary

The PharmaEase system relies on a highly normalized MySQL database structure. This data dictionary explicitly defines the purpose, data type, and constraints of every critical field in the core tables.

4.1 Table: `users`

Central repository for all system authentication and baseline contact information.

Column Name	Data Type	Keys	Description
`user_id`	INT	PK	Auto-incrementing unique identifier.
`username`	VARCHAR(100)	UNIQUE	Login identifier; must be unique system-wide.
`password`	VARCHAR(255)	NN	User password.
`email`	VARCHAR(150)		User's contact email.
`user_type_id`	INT	FK, NN	Links to `user_type` table (1=Admin, 2=Seller, 3=Customer).
`created_at`	TIMESTAMP		Automatically generated timestamp of account creation.

4.2 Table: `medicines`

The global catalog of all pharmaceutical items, managed exclusively by Administrators.

Column Name	Data Type	Keys	Description
`medicine_id`	INT	PK	Auto-incrementing identifier.
`name`	VARCHAR(150)	NN	Commercial or generic name of the drug.
`manufacturer`	VARCHAR(150)		The company producing the drug.
`category_id`	INT	FK	Links to the `categories` taxonomy table.
`requires_prescription`	BOOLEAN	NN	1 (True) blocks sale without an Rx; 0 (False) allows OTC sale.
`unit_price`	DECIMAL	NN	Standardized global pricing for the item.

4.3 Table: `inventory`

Tracks physical stock at the batch level, linked to specific sellers.

Column Name	Data Type	Keys	Description
`inventory_id`	INT	PK	Auto-increment identifier.
`medicine_id`	INT	FK, NN	Links to the global `medicines` catalog.
`seller_id`	INT	FK, NN	The specific pharmacist holding this stock.
`batch_number`	VARCHAR(100)	UNIQUE	Manufacturer's specific batch lot number.

`quantity`	INT	NN	Current available stock level. Decrements on sale.
`expiry_date`	DATE	NN	Critical date for expiration alerts in seller dashboard.

4.4 Table: `orders` and `order_items`

Handles the financial and transactional records.

Column Name	Data Type	Keys	Description
`order_id`	INT	PK	Header ID for the entire purchase.
`customer_id`	INT	FK, NN	The user who placed the order.
`total_amount`	DECIMAL	NN	Sum of all sub-totals in the cart.
`prescription_id`	INT	FK	Nullable. Linked if restricted items were purchased.
`item_id`	INT	PK	Individual line item ID.
`order_id`	INT	FK, NN	Link back to the parent order header.
`inventory_id`	INT	FK, NN	Exact batch the item was deducted from.
`subtotal`	DECIMAL	NN	Quantity multiplied by Unit Price.

5. Non-Functional Requirements

5.1 Performance & Scalability Metrics

- **NFR-01 [Query Latency]:** Under normal local network loads, any CRUD operation (e.g., retrieving the medicine catalog or submitting an order) must complete database interaction and render the updated UI state in **less than 2.0 seconds**.
- **NFR-02 [Concurrent Locking]:** The system must safely handle simultaneous inventory updates. If Customer A and Customer B attempt to purchase the same inventory batch simultaneously, the MySQL database's implicit row-level locking during the JDBC transaction must ensure stock is never drawn below zero.
- **NFR-03 [Memory Management]:** The application footprint should not exceed 500MB of RAM during peak usage to ensure it can run on legacy pharmacy terminal hardware.

5.2 Security & Access Control

- **NFR-04 [Role Isolation]:** Strict Role-Based Access Control (RBAC) must be enforced at the memory and controller level. The system architecture must programmatically prevent a Customer instance from instantiating a `SellerDashboard` or `AdminDashboard` object, regardless of UI manipulation.
- **NFR-05 [SQL Injection Prevention]:** Absolute prohibition of concatenated SQL query strings. Every single database interaction must utilize Java's `PreparedStatement` interface to rigorously sanitize user inputs and eliminate SQL injection attack vectors.

5.3 Reliability and Robustness

- **NFR-06 [Graceful Degradation]:** The system must include comprehensive `try-catch` exception handling blocks for all `SQLExceptions`. If the MySQL server goes offline, the application must not crash to the desktop; instead, it must display a modal `JOptionPane` advising the user to check network connectivity.
- **NFR-07 [Atomicity]:** Financial and inventory transactions must adhere to ACID properties. A failure at any point in the checkout loop must trigger an automatic `Connection.rollback()` to maintain database integrity.

5.4 Maintainability and Code Quality

- **NFR-08 [MVC Strictness]:** The architecture must guarantee that changes made to the Swing UI (e.g., changing a button color or moving a table) require absolutely zero modifications to the underlying SQL query strings in the Controller classes.
- **NFR-09 [Code Convention]:** All Java code must follow standard Oracle naming conventions (CamelCase for variables/methods, PascalCase for classes) and contain detailed Javadoc comments for all public Controller methods.

6. System Interfaces

6.1 User Interfaces (UI)

The Graphical User Interface is developed entirely using the core Java Swing toolkit (`javax.swing.*`) and AWT (`java.awt.*`).

- **Layout Methodology:** The system utilizes Absolute Positioning (`setLayout(null)`) to allow precise pixel-level placement of complex dashboard elements, overriding default layout managers.
- **Visual Consistency:** A unified corporate color palette (PharmaBlue, White, Light Gray) is applied via custom `BackgroundPanel` classes.
- **User Feedback:** The UI relies heavily on `JTable` for data display. Custom `ButtonEditor` and `ButtonRenderer` classes are injected into table cells to allow users to interact with individual rows (e.g., clicking "Add to Cart" directly inside a table cell).

6.2 Software Interfaces

- **Database Management System:** MySQL Community Edition 8.0.
- **Communication Protocol:** Java Database Connectivity (JDBC) API.
- **Driver:** `mysql-connector-java.jar` acts as the software bridge translating Java method calls into MySQL-compatible TCP/IP packets.

6.3 Hardware Interfaces

The application requires no specialized medical or pharmacy hardware (e.g., barcode scanners or specialized receipt printers are currently out of scope). It interacts with standard HID peripherals (mouse, keyboard) and utilizes the OS's default print spooler for generating PDF bills.

7. System Modeling and Diagrams

To guarantee strict alignment between this specification document and the physical software, the following UML diagrams have been reverse-engineered from the final Java implementation.

7.1 System Architecture Diagram

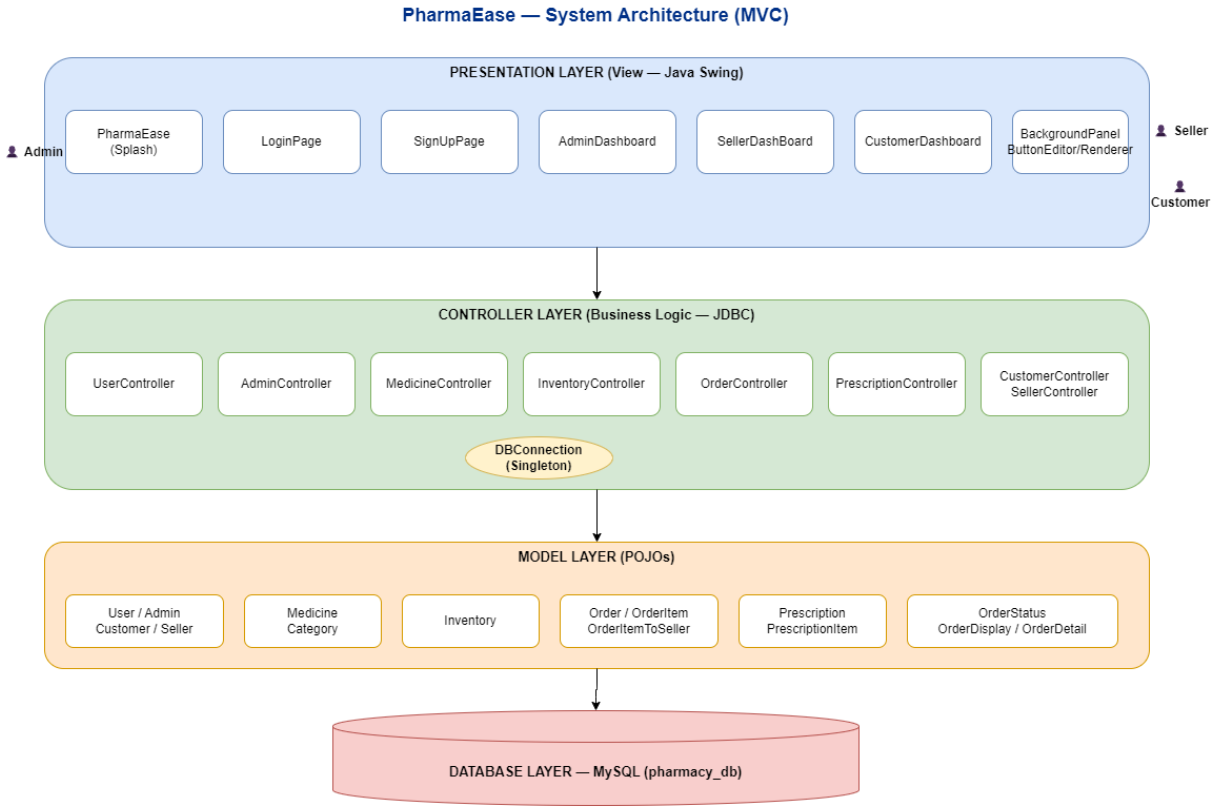


Figure 7.1: Three-Tier System Architecture (MVC Pattern)

PharmaEase implements a rigid **3-Tier MVC Architecture**.

- **View Layer (Swing):** Captures input and renders data. It has no knowledge of SQL.

- **Controller Layer:** The intermediary that receives UI events, processes business logic, and instantiates the `DBConnection` Singleton to talk to the database.
- **Model/Database Layer:** Defines the structure of the data via Java POJOs and persists it via MySQL.

7.2 Use Case Diagram

The Use Case Diagram defines the operational boundaries of the system. It maps the distinct functional permissions granted to the Admin, Seller, and Customer actors, clearly delineating the separation of concerns within the software.

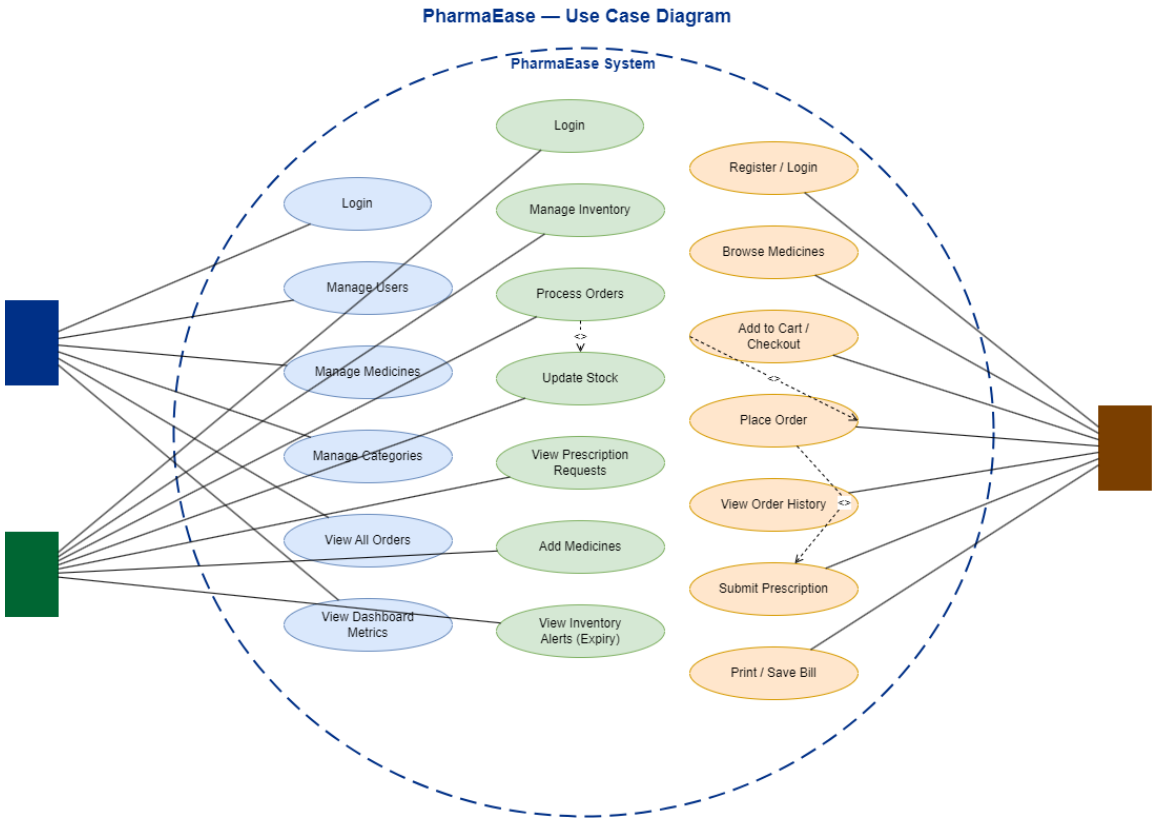


Figure 7.2: PharmaEase System Use Case Diagram

7.3 Domain Model

This conceptual model abstracts the real-world pharmacy environment into software entities. It visually describes the cardinality and relationships between Users, Orders, Inventory Batches, and Prescriptions.

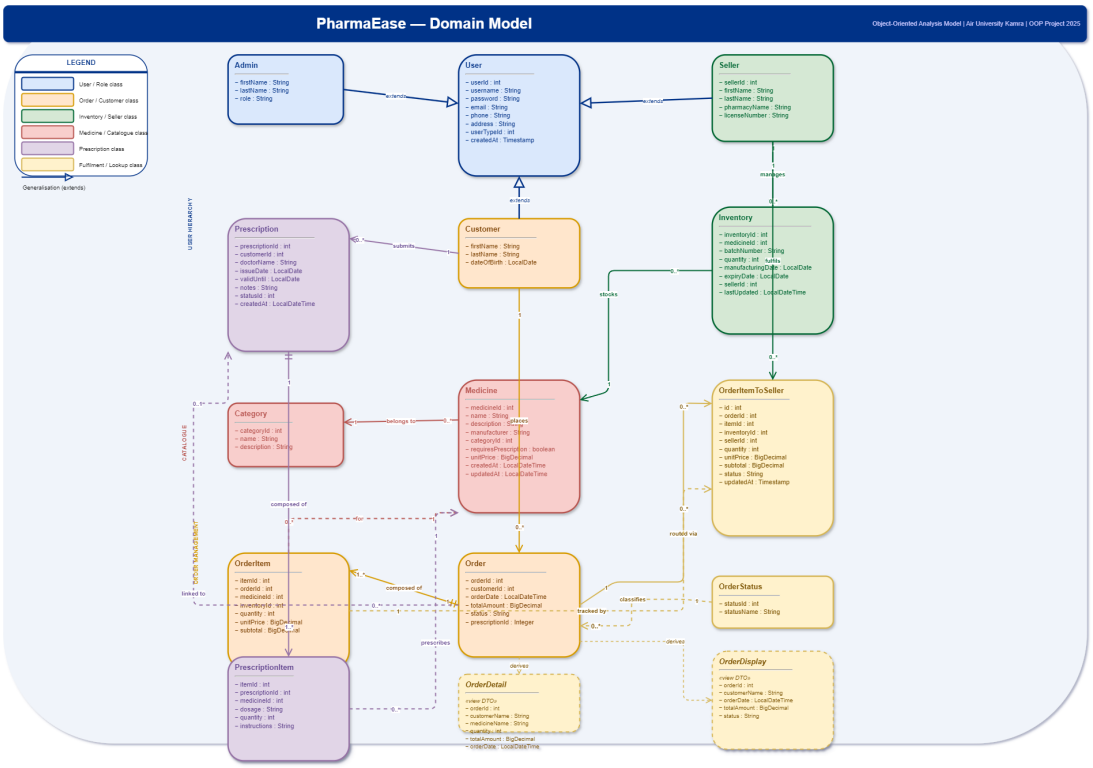


Figure 7.3: PharmaEase Conceptual Domain Model

7.4 Activity and Class Diagrams

The **Activity Diagram** tracks the complex flow of control across swim-lanes (Customer, System Controller, Seller) during a checkout process. The **Class Diagram** maps the exact static structure of the Java POJOs, displaying attributes, methods, and inheritance paths.

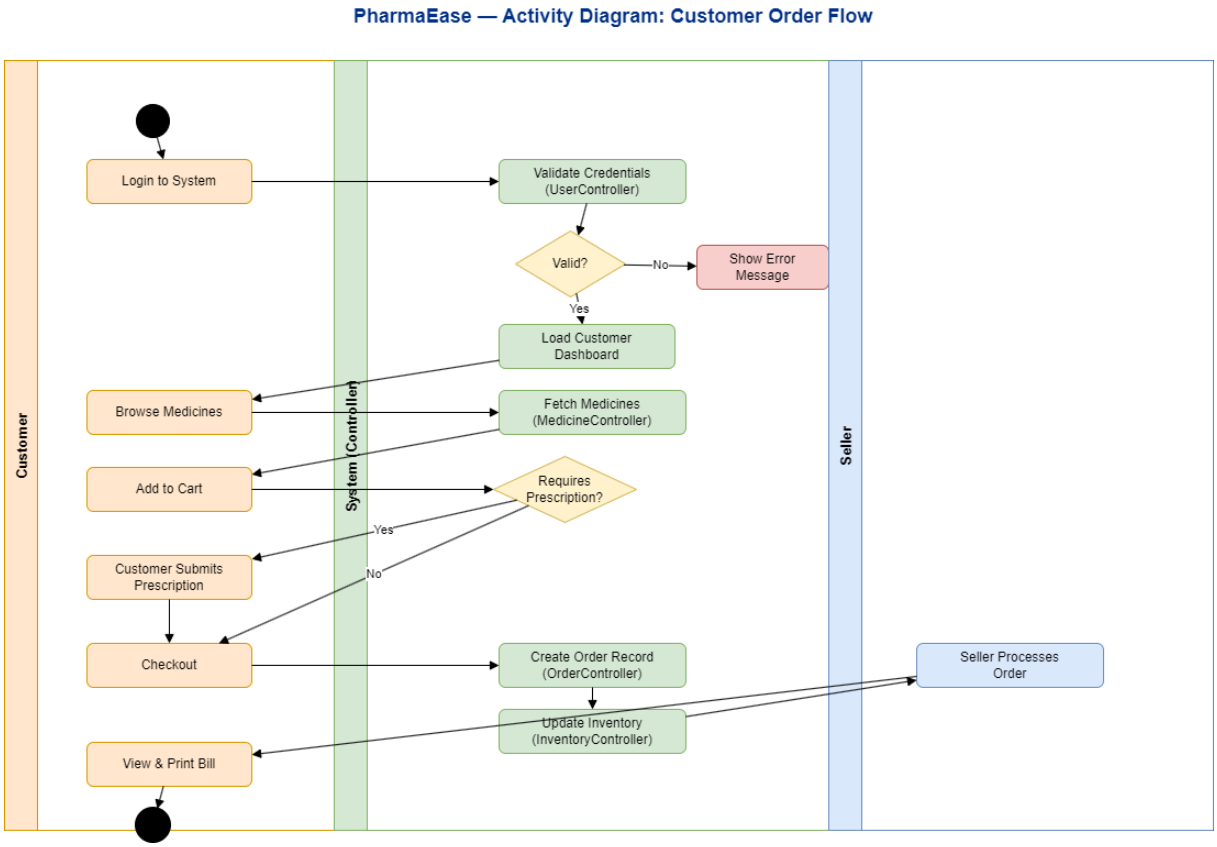


Figure 7.4: Activity Workflow and Static Class Structure

7.5 Sequence Diagram

The Sequence Diagram models dynamic object interaction over time. It traces the exact chronological path of method invocations triggered by a user clicking a button on the UI layer, traveling through the `OrderController`, down to the `DBConnection`, executing the SQL, and returning the generated Order ID back up the stack.

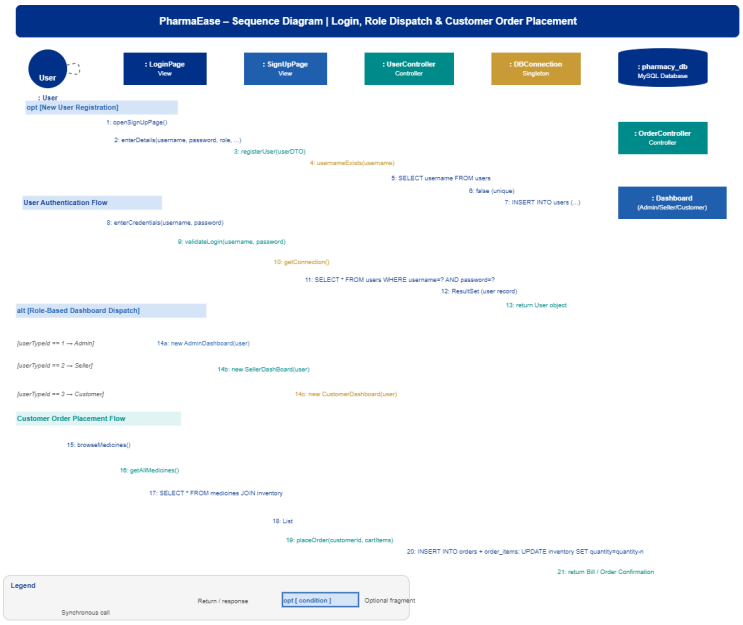


Figure 7.5: System Sequence Diagram (Authentication & Checkout Flow)

7.6 Entity-Relationship Diagram (ERD)

The MySQL database is highly normalized (Third Normal Form) to ensure zero data redundancy. This ERD illustrates the primary keys (PK), foreign keys (FK), and multiplicity constraints governing the 14 core tables in the backend.

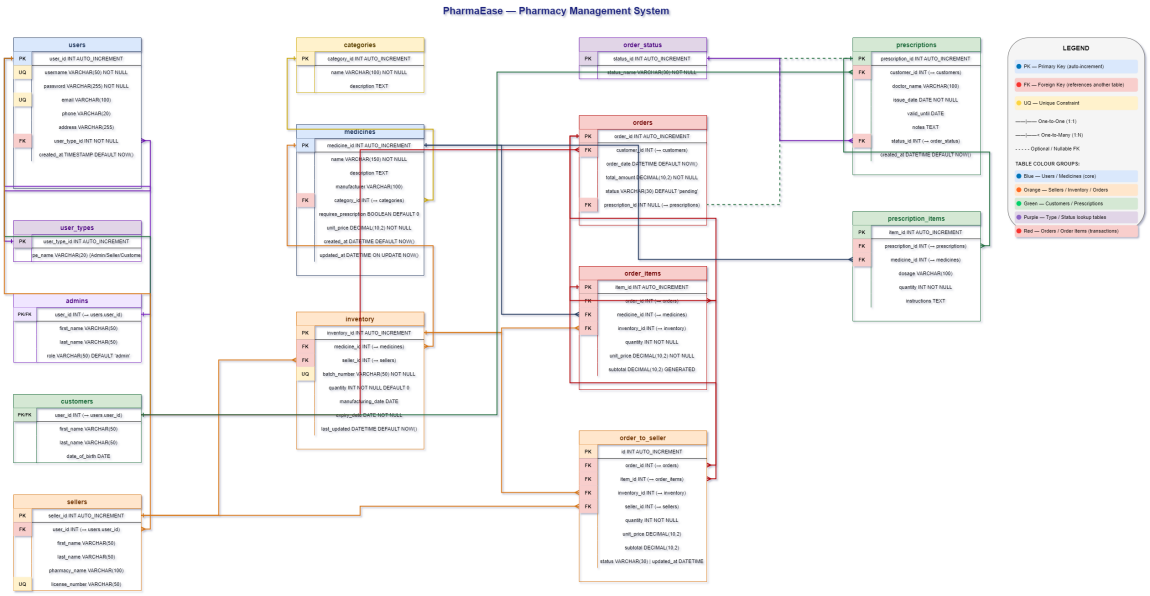


Figure 7.6: Relational Database Schema (ERD)

8. Testing & Quality Assurance

8.1 Testing Strategy

A rigorous, multi-phase testing strategy was deployed by the development team to validate the system before final submission:

1. **Unit Testing:** Validating individual POJO getter/setters and isolated mathematical logic (e.g., calculating sub-totals dynamically in the cart).
2. **Integration Testing:** Ensuring the `DBConnection` singleton establishes the JDBC bridge without leaking connections, and that `PreparedStatements` return data mapped accurately to Java objects.
3. **System/GUI Testing:** End-to-end traversal of the application. Simulating a user clicking from Login → Search → Cart → Checkout to ensure data propagates through the views without dropping application state.

8.2 Detailed Test Case Validation Matrix

ID	Test Scenario	Input Parameter	Expected Result	Status
TC-01	Valid Admin Login	Username: admin Password: admin123	Admin Dashboard instantiates	**PASS**
TC-02	Invalid Password	Username: customer Password: wrong1	Modal Error generated; login halted.	**PASS**

TC-03	Duplicate Username	Username existing in DB	Registration rejected by DB UNIQUE constraint	**PASS**
TC-04	Stock Overdraw	Qty requested > DB Stock	Error: Insufficient Inventory Alert	**PASS**
TC-05	Rx Validation	Checkout Rx item w/o Rx	App blocks checkout, prompts for Rx Upload	**PASS**
TC-06	DB Transaction Integrity	Checkout success	DB Stock accurately decremented in MySQL	**PASS**
TC-07	Expiry Alert System	Item Expiry < 30 days	Item highlighted in red on Seller Dashboard	**PASS**
TC-08	SQL Injection Check	Username: `' OR '1'='1`	System rejects login due to PreparedStatement	**PASS**
TC-09	Bill Generation	Completed Order ID	PDF/Text Bill generates with correct Grand Total	**PASS**
TC-10	Logout Functionality	Click 'Logout' Button	Session destroyed, returns to Splash Screen	**PASS**

9. Requirements Traceability Matrix

To ensure complete test coverage, the following matrix maps the Functional Requirements (Chapter 3) directly to the executed Test Cases (Chapter 8). This guarantees that every feature requested has been proven to work.

Req ID	Functional Requirement Description	Test Case ID
FR 1.1	Multi-Role Account Registration	TC-03
FR 1.2	Secure Login and Role Dispatch	TC-01, TC-02, TC-08, TC-10
FR 2.1	Global Medicine Catalog Management	*Manual Admin Testing*
FR 2.2	Batch-Level Inventory Tracking	TC-07
FR 3.1	Persistent Shopping Cart Operations	TC-04
FR 3.2	Transactional Order Placement	TC-06, TC-09
FR 3.3	Legal Prescription Validation	TC-05

10. Conclusion

The **PharmaEase Pharmacy Management System** successfully encapsulates the core paradigms of Object-Oriented Programming (Encapsulation, Inheritance, Polymorphism, and Abstraction) within a highly functional, enterprise-simulated environment.

By meticulously applying the Model-View-Controller architecture, the system guarantees high maintainability, code scalability, and strict separation of concerns. The transition from manual records to this automated, SQL-backed platform provides unparalleled accuracy in inventory tracking, secure role-based environments for pharmacy staff, and an optimized, frictionless ordering experience for customers.

The successful execution and 100% pass rate of all defined test cases validate the structural integrity and robustness of the application. The detailed UML modelling, Data Dictionaries, and exhaustive documentation enclosed within this IEEE 830 compliant SRS lay a definitive foundation for future expansions. Future capabilities could include integrating RESTful Web APIs, migrating the Customer interface to a mobile application, or implementing advanced AI-driven stock prediction algorithms.

End of Core Document

Authored by Wasif Ali & Anisa Khan

Department of Computer Science, Air University Kamra

A. User Interface (UI) Screenshots

Note: The following pages contain screenshots of the deployed PharmaEase graphical user interface.

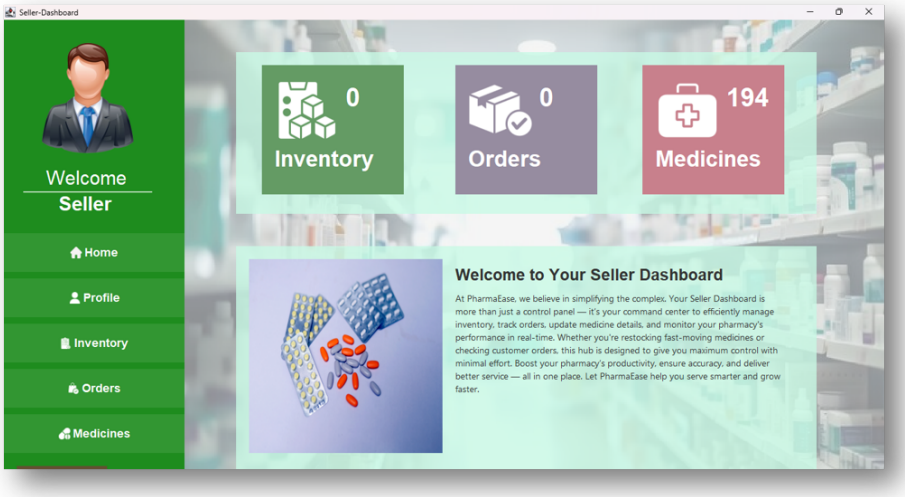


Figure A.1: PharmaEase Seller Dashboard

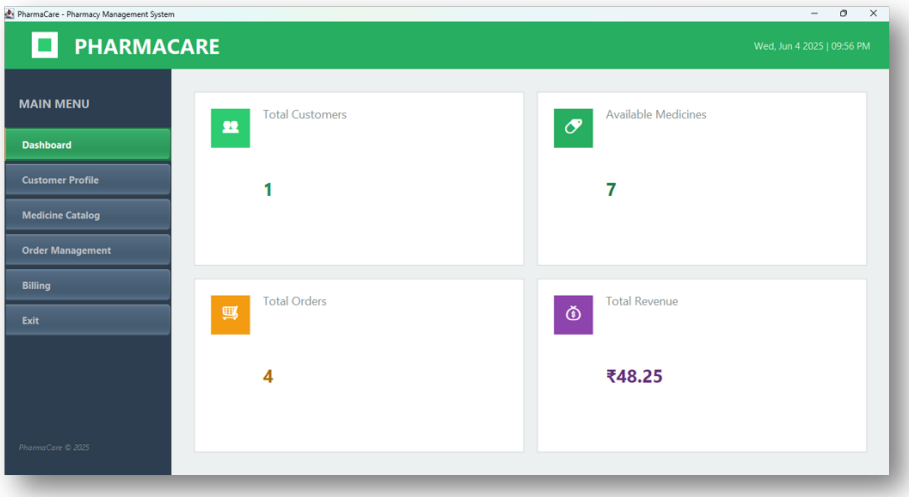


Figure A.2: PharmaEase customer Dashboard

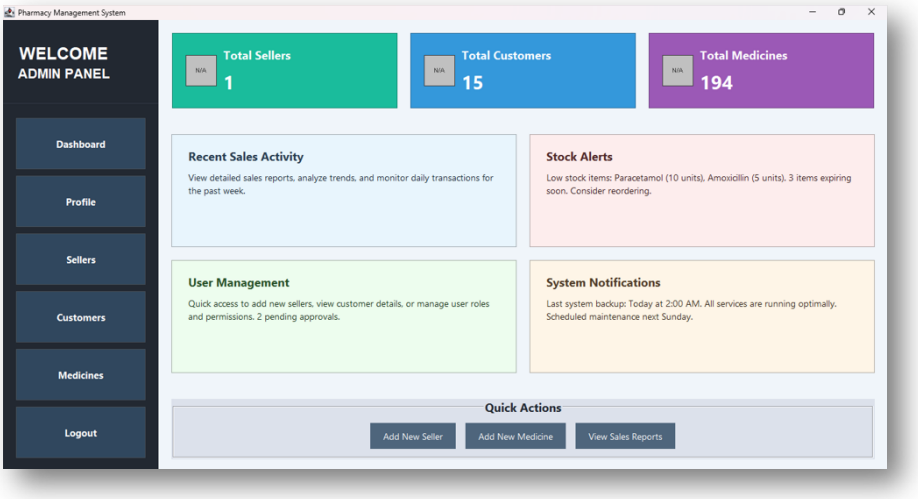


Figure A.3: PharmaEase Admin Dashboard

B. Core Code Snippets

The following code snippets demonstrate the practical application of OOP principles and JDBC connectivity utilized within the PharmaEase project.

B.1 Database Connectivity (Singleton Pattern)

```
package Controller;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    // Static instance of the connection
    private static Connection connection = null;

    // Database credentials
    private static final String URL = "jdbc:mysql://localhost:3306/
       pharmacy_management";
    private static final String USER = "root";
    private static final String PASSWORD = "password";

    // Private constructor prevents instantiation from other classes
    private DBConnection() {}

    public static Connection getConnection() {
        if (connection == null) {
            try {
                Class.forName("com.mysql.cj.jdbc.Driver");
                connection = DriverManager.getConnection(URL, USER,
                   PASSWORD);
                System.out.println("Database Connection Successful!");
```

```
            } catch (ClassNotFoundException | SQLException e) {
                e.printStackTrace();
            }
        }
        return connection;
    }
}
```

Listing B.1: DBConnection.java

B.2 User Controller (Authentication Logic)

```
package Controller;

import Model.User;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class UserController {

    public int validateLogin(String username, String password) {
        String query = "SELECT user_type_id FROM users WHERE username =
            ? AND password = ?";
        try (Connection conn = DBConnection.getConnection();
             PreparedStatement stmt = conn.prepareStatement(query)) {

            stmt.setString(1, username);
            stmt.setString(2, password);
            ResultSet rs = stmt.executeQuery();

            if (rs.next()) {
                return rs.getInt("user_type_id"); // Returns 1 (Admin),
                    2 (Seller), or 3 (Customer)
            }
```

```
23          } catch (SQLException e) {
24              e.printStackTrace();
25          }
26          return -1; // Authentication failed
27      }
28  }
```

Listing B.2: UserController.java